Heaven's Light is Our Guide

Rajshahi University of Engineering & Technology

Department of Computer Science and Engineering

Course Code : CSE 4120
Course Title : Data Mining Sessional

Lab Report

Experiment Name: **Introduction to Data Mining and Tools**

Submission Date: July 21, 2025

Submitted By:

Eazdan Mostafa Rafin
Roll: 2003055
Section : **A**

Submitted To:

Julia Rahman
Associate Professor
Dept. of CSE, RUET

Week 1: Introduction to Data Mining and Tools

Objective

- Understand basic data mining concepts, tasks, and tools.
- Load and explore multiple datasets using Python and libraries (Pandas, scikit-learn, Matplotlib).
- Identify data dimensions, types, and target variables in each dataset.
- Split each dataset into training (60%), validation (20%), and test (20%) sets.
- Visualize label distributions and determine the problem type (classification vs. regression).

Experiments/Problems

- Introduction to Data Mining and Tools.
- Dataset inspection: structure, data types, and label identification.
- Data splitting into train/validation/test (60/20/20).
- Label distribution analysis for determining task type.
- Preparation for downstream machine learning experiments.

1 Introduction

Data mining encompasses several key tasks including *classification*, *clustering*, *association rule mining*, and *regression*. Classification involves predicting a categorical label for each instance, whereas clustering groups similar unlabeled data. Association rule mining (market basket analysis) discovers relationships between items in transaction data. Regression deals with predicting a continuous numeric outcome. Common tools include Weka, Orange, and Python libraries such as Pandas, scikit-learn, and Matplotlib. In this lab, we focus on using Python to analyze data.

The main tasks of this lab are:

- Load each dataset and inspect its structure (dimensions and data types).
- Identify the label (target) variable in each dataset and separate features from labels.

- Split each dataset into training (60%), validation (20%), and test (20%) subsets.
- Count and plot the distribution of labels for classification tasks.
- Determine the type of problem (multiclass classification or regression) based on the label.

The datasets used are:

- **Iris dataset:** 150 samples, 4 numeric features (flower measurements) and 1 categorical label (species: Iris-setosa, Iris-versicolor, Iris-virginica). Used for classification/clustering.
- **Online Retail dataset:** Retail transaction data (InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID, Country). Used for association rule mining or clustering.
- **House Prices dataset:** House attributes with sale prices. Used for regression (predicting house sale price).

2 Iris Dataset Analysis

The Iris dataset consists of 150 observations of iris flowers with four numeric features and a categorical species label. We load the data using Pandas and inspect its structure:

```
print("Iris dataset shape:", iris.shape)
print("Data types of each column in Iris dataset:", iris.dtypes)
print(iris.head())
```

Listing 1: Inspect the Iris dataset

The output shows that the Iris dataset has 150 rows and 5 columns. The first four columns are numeric features (float64), and the last column (`Species`) is an object (string) representing the class label. The printed first few rows confirm this format.

Next, we split the features and label:

```
X_1 = iris.iloc[:, :-1].values
y_1 = iris.iloc[:, -1].values
```

Listing 2: Split Iris features and label

Here, `X_1` contains all four feature columns and `y_1` contains the species label.

We then split the data into training, validation, and test sets (60%/20%/20%):

```
from sklearn.model_selection import train_test_split

X_train_1, X_temp, y_train_1, y_temp = train_test_split(
    X_1, y_1, test_size=0.4, random_state=42
)
X_val_1, X_test_1, y_val_1, y_test_1 = train_test_split(
    X_temp, y_temp, test_size=0.5, random_state=42
)
print(f"Train: {len(X_train_1)}, Validation: {len(X_val_1)}, Test:
   {len(X_test_1)}")
```

Listing 3: Split Iris into train/val/test

The printed output confirms the sizes of each split. For example, with 150 samples, we obtain 90 training, 30 validation, and 30 test instances (60%, 20%, 20%).

To analyze the label distribution, we compute unique labels and their counts:

```
import numpy as np
import matplotlib.pyplot as plt

unique_labels, counts = np.unique(y_1, return_counts=True)
print("Unique labels in y_1:", unique_labels)
print("Counts for each label:", counts)

# Plot label distribution
plt.figure(figsize=(6, 4))
plt.bar(unique_labels, counts, edgecolor='black')
plt.title("Label Distribution in Iris Dataset")
plt.xlabel("Label")
plt.ylabel("Frequency")
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```

Listing 4: Iris label distribution

The output shows three unique labels (`Iris-setosa`, `Iris-versicolor`, `Iris-virginica`) with counts [50, 50, 50], indicating each class has 50 instances. Figure 1 (bar chart) would confirm this balanced distribution. Since the label is categorical with three classes, the Iris dataset represents a multiclass classification problem.

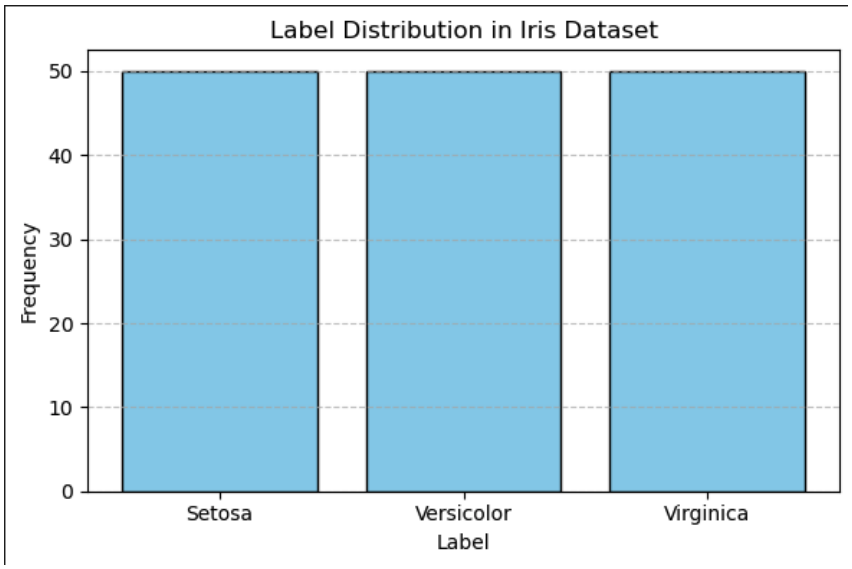

Figure 1: Label distribution in the Iris dataset (bar chart).

3 Housing Dataset Analysis

The House Prices dataset contains various attributes of houses (such as area, bedrooms, etc.) along with the sale price (target). We load the data and inspect it:

```
print("Housing dataset shape:", housing.shape)
print("Data types of each column in housing dataset:",
   housing.dtypes)
print(housing.head())
```

Listing 5: Inspect the Housing dataset

The output shows the number of rows and columns. In our setup, the first column is assumed to be the house price (target), and the remaining columns are features. The printed head verifies this structure.

We separate features and label by taking the first column as `y_2` and the rest as `X_2`:

```
X_2 = housing.iloc[:, 1:].values
y_2 = housing.iloc[:, 0].values
```

Listing 6: Split Housing features and label

We split into training, validation, and test sets:

```
X_train_2, X_temp, y_train_2, y_temp = train_test_split(
    X_2, y_2, test_size=0.4, random_state=42
)
X_val_2, X_test_2, y_val_2, y_test_2 = train_test_split(
    X_temp, y_temp, test_size=0.5, random_state=42
)
print(f"Train: {len(X_train_2)}, Validation: {len(X_val_2)}, Test:
   {len(X_test_2)}")
```

Listing 7: Split Housing into train/val/test

The printed result confirms the 60/20/20 split (e.g., if there were 1460 samples, we get 876 train, 292 validation, 292 test).

Label analysis:

```
unique_labels, counts = np.unique(y_2, return_counts=True)
print("Unique labels in y_2:", unique_labels)
print("Counts for each label:", counts)

# Plot label distribution
plt.figure(figsize=(8, 5))
plt.bar(unique_labels.astype(str), counts, edgecolor='black')
plt.title("Label Distribution in Housing Dataset")
plt.xlabel("Labels")
plt.ylabel("Frequency")
plt.grid(axis='y', linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()
```

Listing 8: Housing label distribution

The target values (house prices) are mostly unique continuous numbers. The printed counts would show many labels with count 1 or small values, reflecting continuous variation. The bar chart in Figure 2 illustrates this wide distribution of prices. Since the label is continuous, this dataset is used for a regression task (predicting sale price).

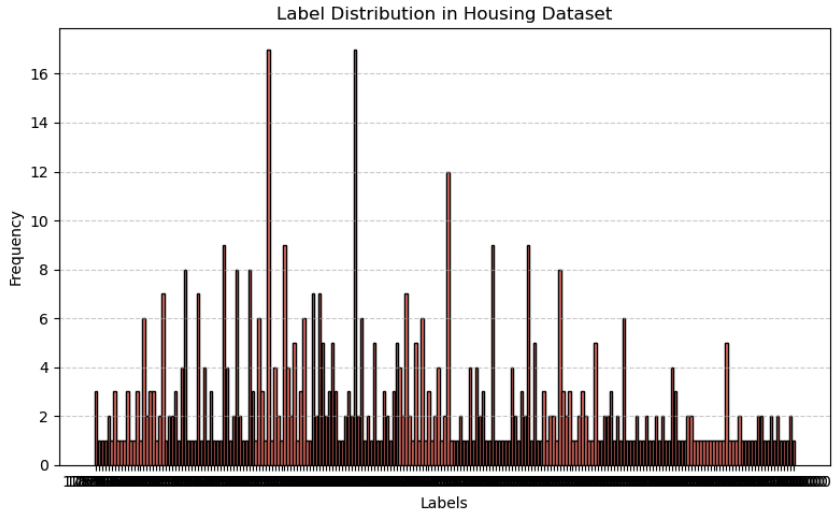

Figure 2: Label distribution in the House Prices dataset (bar chart).

4 Online Retail Dataset Analysis

The Online Retail dataset contains transaction records. It includes columns like InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID, and Country. This dataset is often used for association analysis. We inspect its structure:

```
print("Online Retail dataset shape:", retail.shape)
print("Data types of each column in Online Retail dataset:",
   retail.dtypes)
print(retail.head())
```

Listing 9: Inspect the Online Retail dataset

The output shows a large number of rows (hundreds of thousands) and 8 columns. Data types are mixed: some numeric (Quantity, UnitPrice), some categorical or text (StockCode, Description, Country), and an InvoiceDate.

In the provided code, we move the `UnitPrice` column to the end so it becomes the target:

```
c1 = retail['UnitPrice']
retail = retail.drop(columns=['UnitPrice'])
retail['UnitPrice'] = c1
print(retail.head())
```

Listing 10: Move UnitPrice to last column

After this manipulation, `UnitPrice` is the last column.

We then separate features and label:

```
X_3 = retail.iloc[:, :-1].values
y_3 = retail.iloc[:, -1].values
```

Listing 11: Split Retail features and label

Split into train/validation/test:

```
X_train_3, X_temp, y_train_3, y_temp = train_test_split(
    X_3, y_3, test_size=0.4, random_state=42
)
```

```
X_val_3, X_test_3, y_val_3, y_test_3 = train_test_split(
    X_temp, y_temp, test_size=0.5, random_state=42
)
print(f"Train: {len(X_train_3)}, Validation: {len(X_val_3)}, Test:
   {len(X_test_3)}")
```

Listing 12: Split Retail into train/val/test

This again produces a 60/20/20 split of the data.

Label distribution:

```
unique_labels, counts = np.unique(y_3, return_counts=True)
print("Unique labels in y_3:", unique_labels)
print("Counts for each label:", counts)

# Plot label distribution
plt.figure(figsize=(8, 5))
plt.bar(unique_labels.astype(str), counts, edgecolor='black')
plt.title("Label Distribution in Online Retail Dataset")
plt.xlabel("Labels")
plt.ylabel("Frequency")
plt.grid(axis='y', linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()
```

Listing 13: Retail label distribution

The `UnitPrice` values are continuous (floats), so there are many unique labels. The bar chart in Figure 3 would display the frequency of each unit price (likely mostly count 1 or small). Treating UnitPrice as the target makes this a regression problem.

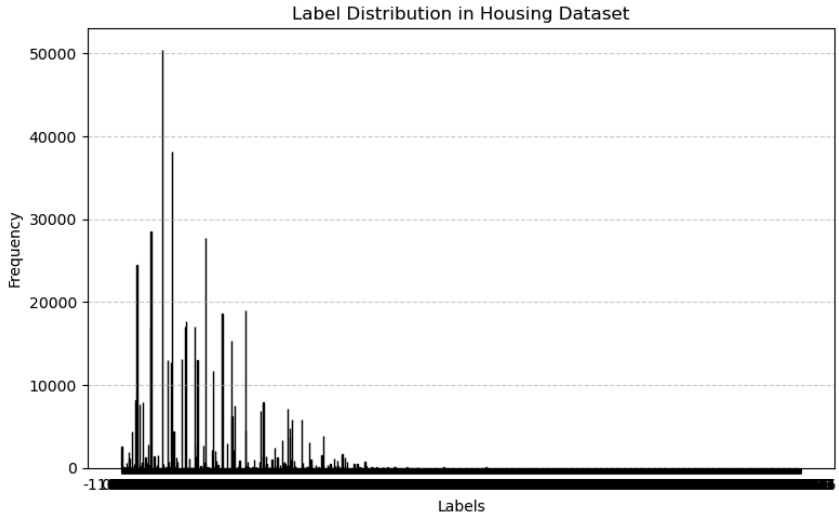

Figure 3: Label distribution in the Online Retail dataset (bar chart).

Conclusion

- **Iris dataset:** 150 samples, 4 features, 3 classes (each with 50 samples). Used for multi-class classification.

- **House Prices:** Continuous sale price as label (regression task). Many unique target values.
- **Online Retail:** After setting `UnitPrice` as label, also a regression task with continuous targets.
- Each dataset was split into 60% training, 20% validation, and 20% test sets.
- Label distributions were plotted (see Figures 1, 2, 3) to confirm problem types.

In summary, Python tools (Pandas, scikit-learn, Matplotlib) were used to load and preprocess the data. The Iris dataset is a balanced multiclass classification problem, while the House Prices and Online Retail datasets are treated as regression problems. This analysis lays the groundwork for applying appropriate machine learning algorithms in future experiments.